python

atec

Single Linked List

A ***Single Linked List*** é uma estrutura de dados linear, onde os elementos são organizados de uma forma sequencial, cada elemento é chamado de nó, contém um valor (o dado) e uma referência (ou ponteiro) para o próximo nó da lista.

È muito utilizada em algoritmos e problemas que requerem manipulação eficiente de dados, como inserções e exclusões, sem a necessidade de realocação de memória, algo comum nas listas nativas de Python.

O primeiro nó é chamado de cabeça (***head***), e o último nó não aponta para nenhum outro, tendo sua referência de próximo nó igual a ***None***, indicando o fim da lista.

Ao contrário de uma lista convencional em Python, que tem complexidade O(n) para operações de inserção e remoção em posições intermediárias, uma lista encadeada simples pode realizar essas operações em O(1) quando o nó de interesse já é conhecido.

Para implementar uma ***single linked list*** em Python, podemos seguir os seguintes passos:

- ✓ ***Definir a estrutura de um nó***: Cada nó armazenará um valor (um caractere) e uma referência para o próximo nó.
- ✓ ***Criar a classe da lista encadeada***: Essa classe terá funções para inserir, remover e exibir os elementos da lista

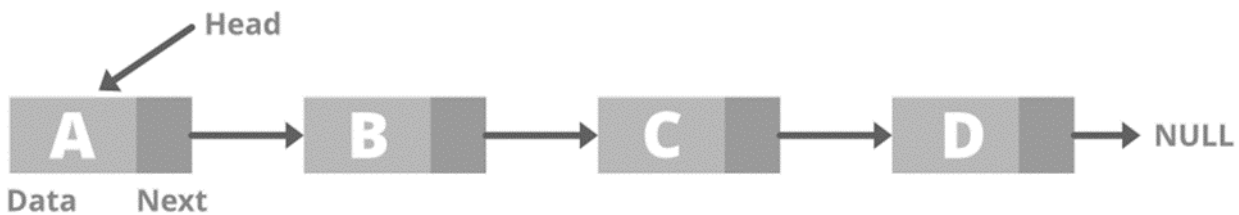
Single Linked list

A ***single linked list*** é uma coleção de nós, onde cada nó contém dois componentes principais:

1)Dados: O valor guardado no nó.

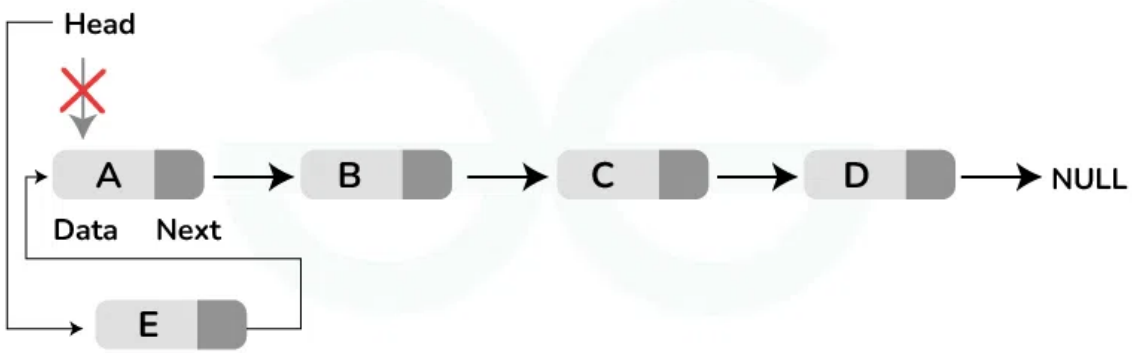
2) Ponteiro: Um ponteiro que aponta para o próximo nó na lista significa que o nó guarda o endereço do proximo nó da sequencia.

- ✓ A lista começa com um nó especial chamado ***HEAD***, que aponta para o primeiro nó da lista.
- ✓ O último nó da lista aponta para ***NULL***, indicando o fim da lista.



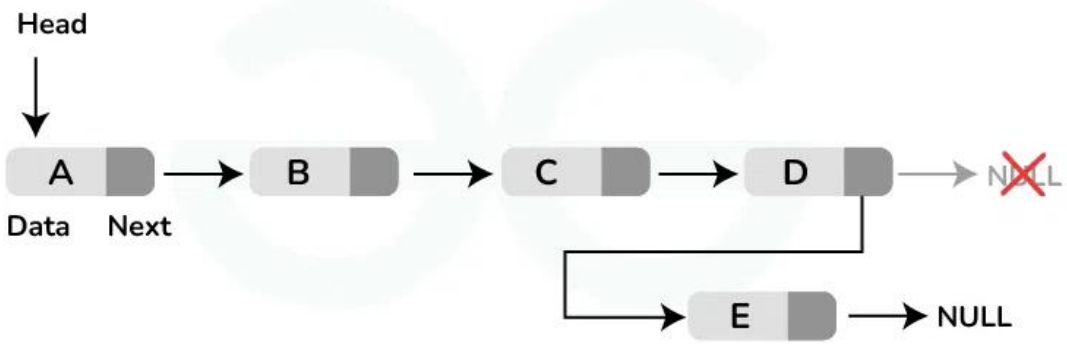
1) Inserir um nó, no inicio da lista

- ✓ Fazer com que o ponteiro (next) do ***novo_nó (E)*** aponte para ***primeiro_nó (A);***
- ✓ Remover o ***head*** do ***primeiro_nó (A)*** original da lista;
- ✓ Fazer com que o ***novo_nó (E)***, tenha o ***head*** da lista.



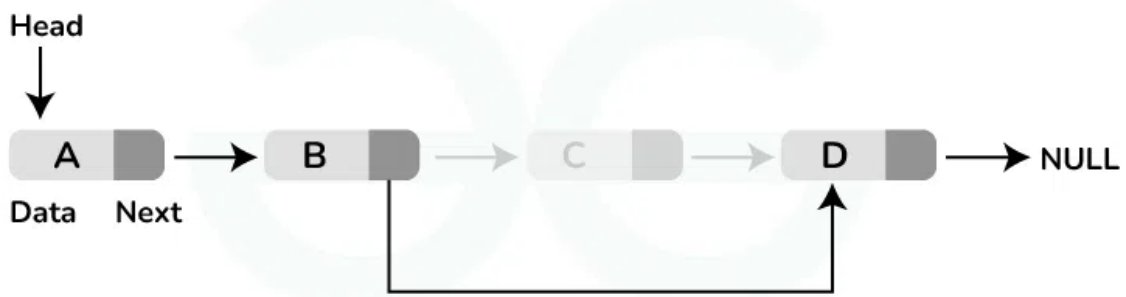
2) Inserir um nó, no fim da lista

- ✓ Tem de iterar toda a lista e se posicionar no ***ultimo_nó (D)***;
- ✓ O ponteiro (next) do ***ultimo_nó (D)***, que está a apontar para ***NULL*** tem de se apontar para o ***novo_nó (E)***;
- ✓ Fazer com que o ponteiro (next) do ***novo_nó*** aponte para ***NULL***.



3) Apagar um nó numa determinada posição

- ✓ Se a lista estiver vazia retorna ***NULL***;
- ✓ Percorrer a lista apartir do ***head*** até encontrar, o ***nó_a_ser_apagado (C)***;
- ✓ Apontar o ponteiro (next) do ***nó_anterior (B)*** para o ***nó_seguinte (D)*** ao que se quer apagar;
- ✓ Apagar o ***nó_a_ser_apagado (C)***.



Vantagens:

- ✓ ***Flexibilidade***: Fácil inserir e remover elementos sem realocação ou reordenação.
- ✓ ***Tamanho Dinâmico***: Não é necessário definir um tamanho no início.

Desvantagens:

- ✓ ***Acesso Sequencial***: Acesso mais lento aos elementos comparado a arrays, pois requer a travessia da lista.
- ✓ ***Overhead de Memória***: Cada nó requer memória adicional para guardar o ponteiro.

https://www.geeksforgeeks.org/singly-linked-list-tutorial/

Estrutura do nó

1) Criar a classe nó no ficheiro ***node.py***

Esta class está sempre presente, independente do tipo de dados que se pretende por na lista.

```
# Definindo a estrutura do nó
class Node:
    def __init__(self, data):
        self.data = data  # Guarda o dado
        self.next = None  # Ponteiro para o próximo nó
```

2) Criar a classe SingleLinkedList, em ***single_linked_list.py***

2.1) Criar o construtor da class

```
from node import Node

# Definindo a classe da lista encadeada simples
class SingleLinkedList:
    def __init__(self):
        self.head = None  # Inicialmente a lista está vazia (head é
None)
```

2.2) Inserir um nó no fim da lista

```
# Inserir um novo caractere no final da lista
    def insert(self, data):
        new_node = Node(data)  # Cria um novo nó com o caractere fornecido
        if self.head is None:  # Se a lista estiver vazia
            self.head = new_node
        else:
            # Percorre a lista até o último nó
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node  # Adiciona o novo nó no final
```

2.3) Inserir um nó numa determinada posição da lista

```
# Inserir um novo caractere em uma posição específica
    def insert_at_position(self, data, position):
        new_node = Node(data)
        if position < 0:
            print("Posição inválida.")
            return
        if position == 0:  # Inserção na posição 0 (início da lista)
            new_node.next = self.head
            self.head = new_node
            return
        current = self.head
        current_position = 0
        previous = None
        # Percorrer até encontrar a posição
        while current and current_position < position:
            previous = current
            current = current.next
            current_position += 1
        if current_position == position:# Inserir na posição desejada
            new_node.next = current
            previous.next = new_node
        else:
            print("Posição fora do alcance da lista.")
```

2.3) Pesquisar se existe um valor na lista

```
# Método para pesquisar um valor na lista
   def search(self, data):
       current = self.head
       while current:
           if current.data == data:
               return True  # Valor encontrado
           current = current.next
       return False  # Valor não encontrado
```

2.4) Remover o primeiro nó que tenha o caracter passado

```
# Remover o primeiro nó que contenha o caractere fornecido
    def remove(self, data):
        current = self.head
        previous = None
        # Verificar se a lista está vazia
        if current is None:
            print("A lista está vazia.")
            return
        # Percorrer a lista em busca do nó a ser removido
        while current and current.data != data:
            previous = current
            current = current.next
        # Se o nó foi encontrado
        if current:
            if previous is None:  # Remover o primeiro nó (head)
                self.head = current.next
            else:
                previous.next = current.next
            print(f"Caractere '{data}' removido.")
        else:
            print(f"Caractere '{data}' não encontrado.")
```

2.5) Remover o nó numa determinada posição da lista

```
# Remover um caractere em uma posição específica
    def remove_at_position(self, position):
        if self.head is None:
            print("A lista está vazia.")
            return
        if position < 0:
            print("Posição inválida.")
            return
        current = self.head
        current_position = 0
        previous = None
        # Se a posição for 0, remover o primeiro nó (head)
        if position == 0:
            self.head = current.next
            return
        # Percorrer a lista até encontrar a posição
        while current and current_position < position:
            previous = current
            current = current.next
            current_position += 1
        if current:  # Se o nó na posição for encontrado
            previous.next = current.next
        else:
            print("Posição fora do alcance da lista.")
```

2.6) Mostrar elementos da lista

```
# Exibir os elementos da lista
    def display(self):
        current = self.head
        if current is None:
            print("A lista está vazia.")
        else:
            while current:
                print(current.data, end=" -> ")
                current = current.next
            print("None")
```

3) Criar uma class Pessoa

Para utilizar como dado no Node

```
# Pessoa.py
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade


    def __str__(self):
        return f"{self.nome} ({self.idade} anos)"
```

4.1) Utilizar o código feito em ***AppStart.py***

```
from single_linked_list import SingleLinkedList
# Exemplo de uso da lista encadeada
lista = SingleLinkedList()
lista.insert('A')
lista.insert('B')
lista.insert('C')
lista.insert('D')
print("Lista inicial:")
lista.display()
# Inserir um caractere na posição 2
print("\nInserindo 'X' na posição 2:")
lista.insert_at_position('X', 2)
lista.display()
print("\nRemovendo o caractere 'C':")
lista.remove('C')
# Remover o caractere da posição 3
print("\nRemovendo o nó da posição 3:")
lista.remove_at_position(3)
lista.display()
# Pesquisar um valor na lista
pesquisar = 'b'
elemento_encontrado = lista.search(pesquisar)
print(f"O valor '{pesquisar}' está {'presente' if elemento_encontrado else 'ausente'} na lista.")
pesquisar = 'd'
found = lista.search(pesquisar)
print(f"O valor '{pesquisar}' está {'presente' if elemento_encontrado else 'ausente'} na lista.")
```

4.2) Utilizar o código da class Pessoa em ***AppStart.py***

```
from single_linked_list import SingleLinkedList
from Pessoa import Pessoa

lista1 = SingleLinkedList()


p1 = Pessoa("Ana", 25)
p2 = Pessoa("Carlos", 30)
p3 = Pessoa("Beatriz", 22)


lista1.insert(p1)
lista1.insert(p2)
lista1.insert(p3)


print("Lista de pessoas: ")
lista1.display()
```